Week 6: Product Review and Rating Management System

Scenario

The e-commerce company wants to collect customer feedback about purchased products. Customers can provide ratings and reviews after completing their orders. The system should store customer opinions, analyze product performance, and identify highly rated products.

Students will create a Product Review and Rating Module connected with the existing Customer and Product modules.

Tasks to be Done

1. Create Review and Rating Tables

```
SQL> CREATE TABLE Review(Review_ID VARCHAR2(7), Customer_ID NUMBER(38), Prod
uct_ID VARCHAR2(7), Rating NUMBER(1), ReviewText VARCHAR2(50), ReviewDate DA
TE, PRIMARY KEY(Review_ID), FOREIGN KEY(Customer_ID) REFERENCES Customer(Cus
tomer_ID), FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID));

Table created.
```

Store Customer Feedback and Ratings

Tasks:

1. Add reviews for purchased products.

```
SQL> INSERT INTO Review VALUES ('R101',101,'11',5,'Beautiful flowers',TO_DAT
E('2026-09-20','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Review VALUES ('R102',102,'12',4,'Fresh and lovely',TO_DATE
('2026-09-21','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Review VALUES ('R103',103,'13',5,'Excellent bouquet',TO_DAT
E('2026-09-22','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Review VALUES ('R104',104,'14',4,'Very fresh flowers',TO_DA
TE('2026-09-23','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Review VALUES ('R105',105,'15',5,'Amazing flower arrangemen
t',TO_DATE('2026-09-24','YYYY-MM-DD'));

1 row created.
```

2. Store customer ratings from 1 to 5 stars.

```
SQL> ALTER TABLE Review ADD CONSTRAINT chk_rating CHECK(Rating BETWEEN 1 AND 5);

Table altered.
```

3. Update review comments if required.

```
SQL> UPDATE Review SET ReviewText='Very beautiful flower arrangement' WHERE Review_ID='R101';

1 row updated.
```

4. Remove inappropriate or invalid reviews.

```
SQL> DELETE FROM Review WHERE Review_ID='R105';

1 row deleted.
```

2. Retrieve Product Review Details

Write SQL queries to:

1.Display all reviews for a product.

```
SQL> SELECT * FROM Review WHERE Product_ID='11';

REVIEW_I CUSTOMER_ID PRODUCT_ RATING REVIEWTEXT                REVIEWDATE
-------- ----------- -------- ------ ------------------------ ------------
R101             101 11            5 Very beautiful flower arr 20-SEP-26
                                       angement
```

5. Display customer name with their reviews.

```
SQL> SELECT C.Customer_Name,R.ReviewText,R.Rating FROM Customer C JOIN Review R ON C.Customer_ID=R.Customer_ID;

CUSTOMER_NAME
--------------------------------------------------------------------------------
REVIEWTEXT                RATING
------------------------- ------
Ananya
Very beautiful flower arr      5
angement

Priya
Fresh and lovely               4

Kaviya
Excellent bouquet              5

CUSTOMER_NAME
--------------------------------------------------------------------------------
REVIEWTEXT                RATING
------------------------- ------
```

6. Find products having maximum reviews.

```
SQL> SELECT Product_ID,COUNT(*) AS Review_Count FROM Review GROUP BY Product_ID ORDER BY Review_Count DESC;

PRODUCT_ REVIEW_COUNT
-------- ------------
13                1
11                1
12                1
14                1
```

7. Display recent customer feedback.

```
SQL> SELECT * FROM Review ORDER BY ReviewDate DESC;

REVIEW_I CUSTOMER_ID PRODUCT_ RATING REVIEWTEXT                          REVIEWDATE
-------- ----------- -------- ------ ------------------------------------ ----------
R104             104 14            4 Very fresh flowers                   23-SEP-26
R103             103 13            5 Excellent bouquet                    22-SEP-26
R102             102 12            4 Fresh and lovely                     21-SEP-26
R101             101 11            5 Very beautiful flower arrangement    20-SEP-26
```

8. Retrieve reviews with ratings above 4.

```
SQL> SELECT * FROM Review WHERE Rating>4;

REVIEW_I CUSTOMER_ID PRODUCT_ RATING REVIEWTEXT                          REVIEWDATE
-------- ----------- -------- ------ ------------------------------------ ----------
R101             101 11            5 Very beautiful flower arrangement    20-SEP-26
R103             103 13            5 Excellent bouquet                    22-SEP-26
```

3. Calculate Average Product Ratings

Use aggregate functions to analyze ratings.

Tasks:

9. Calculate average rating for each product.

```
SQL> SELECT Product_ID,AVG(Rating) AS Average_Rating FROM Review GROUP BY Product_I;

PRODUCT_ AVERAGE_RATING
-------- --------------
13                    5
14                    4
12                    4
11                    5
```

10. Co unt the number of reviews per product.

```
SQL> SELECT Product_ID,COUNT(*) AS Number_of_Reviews FROM Review GROUP BY Product_ID;

PRODUCT_ NUMBER_OF_REVIEWS
-------- -----------------
13                       1
14                       1
12                       1
11                       1
```

11. Find highest-rated products.

```
SQL> SELECT Product_ID,AVG(Rating) AS Average_Rating FROM Review GROUP BY Product_ID ORDER BY Average_Rating DESC;

PRODUCT_ AVERAGE_RATING
-------- --------------
13                    5
11                    5
12                    4
14                    4
```

12. Identify products with average rating above 4.

```
SQL> SELECT Product_ID,AVG(Rating) AS Average_Rating FROM Review GROUP BY Product_ID HAVING AVG(Rating) 4;

PRODUCT_ AVERAGE_RATING
-------- --------------
13                    5
11                    5
```

4. Identify Highly Rated Products

Generate product rating reports.

Report 1: Product Rating Analysis

Display:

- Product name
- Number of reviews
- Average rating

```
SQL> SELECT Rating,COUNT(*) AS Rating_Count FROM Review GROUP BY Rating ORDER BY Rating DES( ;

RATING RATING_COUNT
------ ------------
     5            2
     4            2
```

Report 2: Customer Feedback Analysis

Find:

- Most reviewed products
- Highly rated products
- Products requiring improvement

```
SQL> SELECT Product_ID,AVG(Rating) AS Average_Rating FROM Review GROUP BY Product_ID HAVING AVG(Rating) 4;

PRODUCT_ AVERAGE_RATING
-------- --------------
13                    5
11                    5
```

Report 3: Rating Distribution

- Number of 5-star ratings
- Number of 4-star ratings

- Number of low-rated products
- Complete Rating